

# PROGRAMAÇÃO DE BANCO DE DADOS

Prof. Rafael Tápio

# LINGUAGEM DE MANIPULAÇÃO DE DADOS

# INSERÇÃO DE DADOS

The background of the slide features a decorative graphic consisting of large, interlocking geometric shapes in orange and white. On the left, there is a solid orange vertical band. To its right, a white shape with a jagged, sawtooth-like edge separates it from another orange vertical band on the far right. The text 'INSERÇÃO DE DADOS' is positioned in the upper left, within the first orange band.

# Estrutura do INSERT

A sintaxe básica do comando INSERT é a seguinte:

```
INSERT INTO nome_da_tabela (coluna1, coluna2, coluna3, ...)  
VALUES (valor1, valor2, valor3, ...);
```

- **nome\_da\_tabela**: O nome da tabela onde os dados serão inseridos.
- **coluna1, coluna2, coluna3, ...**: Os nomes das colunas da tabela onde os dados serão inseridos.
- **valor1, valor2, valor3, ...**: Os valores a serem inseridos nessas colunas.

# Exemplo Básico de INSERT

Vamos imaginar que temos uma tabela chamada clientes com as seguintes colunas:

- id\_cliente (INT)
- nome (VARCHAR)
- email (VARCHAR)
- telefone (VARCHAR)

A tabela pode ser criada com a seguinte estrutura:

```
CREATE TABLE clientes (  
    id_cliente INT AUTO_INCREMENT PRIMARY KEY,  
    nome VARCHAR(100),  
    email VARCHAR(100),  
    telefone VARCHAR(20)  
);
```

# Exemplo Básico de INSERT

Agora, vamos inserir um cliente na tabela clientes:

```
INSERT INTO clientes  
(nome, email, telefone)  
VALUES  
( 'João Silva', 'joao@exemplo.com', '1234-5678' );
```

Este comando insere um novo registro na tabela **clientes**, atribuindo valores para as colunas nome, email e telefone.

Note que a coluna **id\_cliente** não precisa ser especificada, pois é uma chave primária com **incremento automático**.

# Inserção de Múltiplos Registros

É possível inserir vários registros em uma única instrução INSERT. Para isso, basta separar cada conjunto de valores com vírgulas.

```
INSERT INTO clientes (nome, email, telefone)
VALUES
('Maria Oliveira', 'maria@exemplo.com', '9876-5432'),
('Carlos Pereira', 'carlos@exemplo.com', '6543-2109'),
('Fernanda Souza', 'fernanda@exemplo.com', '4321-0987');
```

Este comando insere três registros de uma vez na tabela clientes.

A vantagem dessa abordagem é a performance, pois reduz o número de comandos SQL enviados ao banco.



# Inserção Sem Especificar Colunas

Se você deseja inserir valores para todas as colunas de uma tabela, você pode omitir a lista de colunas. A ordem dos valores deve seguir a mesma ordem das colunas na tabela.

```
INSERT INTO  
clientes  
VALUES  
(NULL, 'Pedro Santos', 'pedro@exemplo.com', '1122-3344');
```

Neste caso, a coluna **id\_cliente** pode ser preenchida com **NULL**, pois ela é auto-incrementada.

As colunas restantes são preenchidas com os valores correspondentes para nome, email e telefone.

# Inserção com Subconsulta

É possível também utilizar subconsultas dentro de um INSERT para inserir dados de outra tabela.

Imagine que você tem uma tabela **novos\_clientes** e quer inserir esses dados na tabela clientes:

```
INSERT INTO clientes (nome, email, telefone)
SELECT nome, email, telefone FROM novos_clientes;
```

A subconsulta **SELECT nome, email, telefone FROM novos\_clientes** recupera os dados da tabela **novos\_clientes**, e esses dados são inseridos na tabela clientes.

Essa abordagem é útil quando você precisa **migrar** ou transferir dados entre tabelas.

# Inserção com Condição **ON DUPLICATE KEY UPDATE**

Se você tentar inserir um registro com uma chave primária que já existe, o banco de dados retornará um erro.

No entanto, você pode usar a cláusula **ON DUPLICATE KEY UPDATE** para **atualizar o registro existente**, em vez de gerar um erro.

Se o id\_cliente já existir, os valores de nome, email e telefone serão atualizados com os novos valores.

Se o id\_cliente não existir, um novo registro será inserido.

```
CREATE TABLE clientes (  
    id_cliente INT AUTO_INCREMENT PRIMARY KEY,  
    nome VARCHAR(100),  
    email VARCHAR(100) UNIQUE,  
    telefone VARCHAR(20)  
);
```

```
INSERT INTO clientes  
(id_cliente, nome, email, telefone)  
VALUES  
(1, 'João Silva', 'joao@novoemail.com', '1234-5678')  
ON DUPLICATE KEY UPDATE  
    nome = VALUES(nome),  
    email = VALUES(email),  
    telefone = VALUES(telefone);
```

# EXERCÍCIOS

## [INSERT]

## Exercício 1 - Inserção Básica

Crie a tabela **produtos** com as colunas **id\_produto**, **nome**, **preco** e **estoque**.

Insira três registros de produtos utilizando a sintaxe básica do INSERT.

## Exercício 2 - Inserção Múltipla

Crie a tabela vendedores com as colunas id\_vendedor, nome, salario e cidade.

Insira cinco vendedores em uma única instrução INSERT.

## Exercício 3 - Inserção com Subconsulta

Crie a tabela **compras** com as colunas **id\_compra**, **id\_cliente**, **id\_produto**, **quantidade** e **data\_compra**.

Crie a tabela **compras\_temporarias** com as mesmas colunas e insira alguns registros.

Transfira os dados de **compras\_temporarias** para a tabela **compras** usando uma **subconsulta**.



## Exercício 4 - Inserção com Atualização

Crie a tabela estoque com as colunas **id\_produto**, **nome\_produto**, **quantidade**.

Insira um registro na tabela estoque e, em seguida, insira outro registro com o mesmo **id\_produto**, mas com valores diferentes. Utilize a cláusula **ON DUPLICATE KEY UPDATE** para atualizar o estoque.

# ATUALIZAÇÃO DE DADOS



# Sintaxe Básica do UPDATE

A estrutura básica do comando UPDATE é a seguinte:

```
UPDATE nome_da_tabela  
SET coluna1 = valor1, coluna2 = valor2, ...  
WHERE condicao;
```

- **nome\_da\_tabela**: O nome da tabela onde os dados serão atualizados.
- **coluna1, coluna2, ...**: As colunas a serem atualizadas.
- **valor1, valor2, ...**: Os novos valores para as colunas.
- **WHERE condicao**: A condição que define quais registros devem ser atualizados. (Se não houver WHERE, todos os registros da tabela serão atualizados!)

## Exemplo de UPDATE Simples

Atualizando o telefone de um cliente:

```
UPDATE clientes  
SET telefone = '4321-8765'  
WHERE id_cliente = 1;
```

Este comando atualiza o telefone do cliente com id\_cliente igual a 1 para o valor '4321-8765'.

# Atualizando Vários Campos

É possível atualizar mais de uma coluna ao mesmo tempo em uma única instrução UPDATE.

Exemplo: Atualizando o nome e o email

**UPDATE clientes**

**SET nome = 'Carlos Eduardo', email = 'carlos.edu@exemplo.com'**

**WHERE id\_cliente = 2;**

Neste exemplo, tanto o **nome** quanto o **email** do cliente com id\_cliente = 2 são atualizados.

# Atualização com Condição IN

O operador IN pode ser usado em conjunto com o UPDATE para atualizar registros que atendem a uma condição que envolve múltiplos valores, ou seja, quando você quer atualizar registros que correspondem a qualquer um de um conjunto de valores.

**UPDATE**

**clientes**

**SET**

**nome = 'Carlos Eduardo'**

**,email = 'carlos.edu@exemplo.com'**

**WHERE**

**id\_cliente IN (2, 3, 5);**

## Atualização com Subconsulta

Agora, queremos atualizar o preço dos produtos que pertencem à categoria "Tecnologia" para um valor mais alto.

```
UPDATE produtos
SET preco = preco * 1.1
WHERE id_categoria = (SELECT id_categoria FROM
categorias WHERE nome_categoria =
'Tecnologia');
```

Este comando atualiza o preço dos produtos da categoria "**Tecnologia**", aumentando-o em **10%**.

# EXERCÍCIOS [UPDATE]



## Exercício 1: Atualização Simples

Crie a tabela funcionários com as colunas id\_funcionario, nome, salario, departamento.

Insira 3 registros, com salários de 1000, 2000 e 3000.

Atualize o salário do funcionário com id\_funcionario = 3 para 3500.00.

## Exercício 2: Atualização de Vários Campos

Crie a tabela fornecedores com as colunas id\_fornecedor, nome, cidade, email.

Insira 5 registros

Atualize o nome e o email do fornecedor com id\_fornecedor = 5.

## Exercício 3: Atualização com Condição IN

Crie a tabela alunos com as colunas id\_aluno, nome, nota\_final.

Insira 4 registros

Atribua uma nota final de 10 para os alunos com id\_aluno igual a 1, 2 ou 3.

## Exercício 4: Atualização com Subconsulta

Crie a tabela categorias com as colunas id (inteiro) e descricao (texto).  
Insira 2 registros na tabela categorias:

- id = 1, descricao = 'Alimentos'
- id = 2, descricao = 'Bebidas'

Crie a tabela produtos com as colunas id\_produto (inteiro), nome\_produto (texto), preco (decimal), e id\_categoria (inteiro, chave estrangeira referenciando a tabela categorias). Insira 3 registros na tabela produtos:

- 2 produtos com id\_categoria = 1
- 1 produto com id\_categoria = 2

Atualize o preço de todos os produtos da categoria "Alimentos", aplicando um desconto de 10% no preço, utilizando uma subconsulta.

# EXCLUSÃO DE DADOS



# Sintaxe Básica do DELETE

A estrutura básica do comando DELETE é a seguinte:

```
DELETE FROM nome_da_tabela  
WHERE condicao;
```

- **nome\_da\_tabela**: O nome da tabela da qual os dados serão excluídos.
- **condicao**: A condição que determina quais registros serão excluídos.  
**Sem a cláusula WHERE, todos os registros da tabela serão excluídos, o que pode ser perigoso.**

## Exemplo de DELETE Simples

Queremos excluir um cliente específico da tabela clientes:

```
DELETE FROM clientes  
WHERE id_cliente = 1;
```

Esse comando exclui o registro do cliente com id\_cliente = 1.

Atenção: Se você omitir a cláusula WHERE, todos os registros da tabela serão excluídos!

```
DELETE FROM clientes;
```

O comando acima excluirá todos os registros da tabela clientes, mas a estrutura da tabela permanecerá.

# Excluindo Múltiplos Registros

É possível excluir múltiplos registros de uma vez, utilizando condições mais amplas com a cláusula WHERE.

Exemplo: Excluindo vários clientes de uma cidade específica

```
DELETE FROM clientes  
WHERE cidade = 'São Paulo';
```

Este comando excluirá todos os registros da tabela clientes onde a cidade é igual a "São Paulo".

Você pode usar diferentes operadores nas condições, como:

**=, >, <, >=, <=:** Operadores de comparação.

**IN, BETWEEN, LIKE:** Operadores para condições mais complexas.



## Excluindo Registros com IN

Às vezes, é necessário excluir vários registros específicos com base em uma lista de valores. A cláusula IN pode ser utilizada para isso.

Exemplo: Excluindo múltiplos clientes com IN

```
DELETE FROM clientes  
WHERE id_cliente IN (1, 3, 5);
```

Este comando exclui os clientes com id\_cliente igual a 1, 3 ou 5.

# Excluindo Registros com Subconsulta

Em alguns casos, você pode precisar usar uma subconsulta para excluir registros com base nos dados de outra tabela.

Exemplo: Excluindo clientes com base em outra tabela

```
DELETE FROM clientes  
WHERE id_cliente NOT IN  
      (SELECT id_cliente FROM pedidos);
```

Esse comando exclui todos os clientes que não têm pedidos registrados na tabela pedidos.

# Excluindo Todos os Registros com TRUNCATE

O comando DELETE pode ser utilizado para remover registros de forma seletiva.

Porém, se você quiser remover todos os registros de uma tabela de maneira mais eficiente (sem registrar as exclusões uma por uma), pode usar o comando TRUNCATE.

Diferenças entre DELETE e TRUNCATE:

- **DELETE:** Pode excluir registros com base em condições e pode ser revertido (com transações).
- **TRUNCATE:** Exclui todos os registros de uma tabela e não pode ser revertido, sendo mais rápido que o DELETE.

# Excluindo Todos os Registros com TRUNCATE

Exemplo de TRUNCATE

```
TRUNCATE TABLE clientes;
```

Esse comando exclui todos os registros da tabela clientes, mas mantém a estrutura da tabela intacta.

**Importante:** Embora o TRUNCATE seja mais rápido, ele não registra cada exclusão individualmente e não pode ser revertido.

# EXERCÍCIOS

## [DELETE]

## Exercício : Exclusão Simples

Crie a tabela funcionarios com as colunas id\_funcionario, nome, salario, departamento.

Exclua o funcionário com id\_funcionario = 3.

## Exercício : Exclusão com Condição

Crie a tabela produtos com as colunas id\_produto, nome\_produto, preco, estoque.

Exclua todos os produtos que possuem estoque igual a 0.

## Exercício : Exclusão com IN

Crie a tabela clientes com as colunas id\_cliente, nome, cidade.  
Exclua os clientes com id\_cliente igual a 1, 4 e 6.



## Exercício : Exclusão com Subconsulta

Crie a tabela alunos com as colunas id\_aluno, nome, nota\_final.

Exclua os alunos que têm nota final inferior à média de todos os alunos, utilizando uma subconsulta.

## Exercício: Truncate

Crie a tabela cursos, com as colunas id\_curso e descricao.

Insira 5 registros

Utilize o comando truncate para apagar todos os registros